

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PURWANCHAL CAMPUS

A PROJECT REPORT ON DISTINGUISHING HUMAN AND AI-GENERATED NEPALI TEXT USING EXPLAINABLE AI

BY

MILAN ARJEL (PUR078BCT050)

SAMIR BIDARI (PUR078BCT073)

SANDEEP POUDEL (PUR078BCT075)

SANDESH POKHAREL (PUR078BCT076)

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

PURWANCHAL CAMPUS

DHARAN, NEPAL

MARCH, 2026

DISTINGUISHING HUMAN AND AI-GENERATED NEPALI TEXT USING EXPLAINABLE AI

By

MILAN ARJEL (PUR078BCT050)

SAMIR BIDARI(PUR078BCT073)

SANDEEP POUDEL(PUR078BCT075)

SANDESH POKHAREL(PUR078BCT076)

Project Supervisor

Asst. Prof. Bikram Shah

A project submitted to the Department of Electronics and Computer Engineering in partial fulfillment of the requirements for the Bachelor's Degree in Computer Engineering

Department of Electronics and Computer Engineering
Purwanchal Campus, Institute of Engineering
Tribhuvan University
Dharan, Nepal

March, 2026

COPYRIGHT©

The author has agreed that the Library, Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the supervisor(s) who supervised the thesis work recorded herein or, in their absence, by the Head of the Department wherein the thesis report was done. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering in any use of the material of this thesis report. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering and author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head

Department of Electronics and Computer Engineering

Purwanchal Campus, Institute of Engineering

Dharan , Sunsari

Nepal

DECLARATION

We hereby declare that this work submitted for Bachelors of Engineering in Computer Engineering at Institute of Engineering, Purwanchal Campus entitled **"DISTINGUISHING HUMAN AND AI-GENERATED NEPALI TEXT USING EXPLAINABLE AI"** is our original work and has been carried out by us under the guidance of Mr. Bikram Shah.

We further declare that this work has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

All sources of information used in this work have been duly acknowledged.

MILAN ARJEL (PUR078BCT050)
SAMIR BIDARI (PUR078BCT073)
SANDEEP POUDEL (PUR078BCT075)
SANDESH POKHAREL (PUR078BCT076)

March, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported us in the completion of this work.

First and foremost, we extend our heartfelt thanks to the Department of Computer Engineering at IOE Purwanchal Campus for providing us with the resources, facilities, and academic environment to successfully complete this project. We are deeply grateful to our Department Head, Mr. Manoj Kumar Guragain, his encouragement and leadership have greatly contributed to the successful completion of this project.

We are also deeply thankful to our supervisor, Mr. Bikram Shah, for his valuable guidance, continuous encouragement, and constructive suggestions throughout the course of this project. His insight and expertise greatly contributed to shaping this work.

We would also like to extend our thanks to our teachers and the faculty members for providing us with the necessary knowledge and resources. Their support has been instrumental in completing this study.

We are grateful to our friends and colleagues for their help, motivation, and for creating a positive environment during this journey.

Finally, we would like to express our heartfelt thanks to our families for their unconditional support, patience, and encouragement, which made it possible for us to complete this work successfully.

MILAN ARJEL (PUR078BCT050)
SAMIR BIDARI (PUR078BCT073)
SANDEEP POUDEL (PUR078BCT075)
SANDESH POKHAREL (PUR078BCT076)

ABSTRACT

The rapid growth of Large Language Models (LLMs) has created a strong need for reliable methods to distinguish between human-written and machine-generated Nepali text. This project implemented a robust classification framework that balances high predictive performance with model interpretability. The core architecture utilized an ensemble modeling approach, strategically weight-combining fine-tuned Transformer models, specifically Nepberta, Multilingual MiniLM and XLM-RoBERTa and Traditional ML models to achieve enhanced robustness across diverse Nepali linguistic contexts. To address the ”black-box” nature of these neural networks, SHAP (SHapley Additive exPlanations) was integrated as an Explainable AI (XAI) component. This helps identify which Nepali words or tokens influenced the model's predictions. Experimental results demonstrated that the ensemble approach outperformed individual base models in both accuracy and F1-score. This system provides a transparent and verifiable tool for the Nepali digital ecosystem, effectively supporting academic integrity and offering a defense against AI-driven misinformation.

Keywords: LLM, Ensemble Learning, SHAP, Explainable AI, Nepali NLP, Classification, AI Detection

TABLE OF CONTENTS

COPYRIGHT© i

DECLARATION ii

ACKNOWLEDGEMENT iii

ABSTRACT iv

LIST OF FIGURES x

LIST OF TABLES xi

LIST OF ABBREVIATIONS xii

1 INTRODUCTION 1

1.1 Background 1

1.2 Gap Identification 1

1.3 Motivation 1

1.4 Objectives 2

2 RELATED THEORY 3

2.1 Natural Language Processing 3

2.2 Transformer 3

2.3 Language Models 4

2.3.1 Foundation Models 4

2.3.2 The Fine-Tuning Process 5

2.4 Machine Learning Classifiers 6

2.4.1 Support Vector Classifier (SVC) 6

2.4.2 Logistic Regression 7

2.4.3 Naive Bayes 7
2.4.4 Random Forest 8
2.4.5 XGBoost 8
2.5 Tokenization 9
2.5.1 Strategies for Tokenization 9
2.6 Embeddings and Vectorization 9
2.6.1 Embeddings 9
2.6.2 Vectorization 10
2.7 Optimization and Loss Functions 10
2.7.1 Cross-Entropy Loss 10
2.7.2 Adam Optimizer 11
2.8 Explainable AI (XAI) 11
2.8.1 SHAP (SHapley Additive exPlanations) 11
2.9 Evaluation Metrics 11
2.9.1 Accuracy, Precision, and Recall 12
2.9.2 Macro-F1 Score 12
2.9.3 Weighted F1 Score 12
2.9.4 Matthews Correlation Coefficient (MCC) 13
2.9.5 Area Under the ROC Curve (AUC) 13
2.10 Ensemble Learning Architectures 13
2.10.1 Voting Mechanisms 14
2.10.2 Bagging and Boosting 14
2.10.3 Stacked Generalization 14

3 LITERATURE REVIEW **16**

4 METHODOLOGY **18**

4.1 Overview 18

4.2 Software Development Model 18

4.3 Research and Problem Analysis 18

4.4 Training Pipeline 19

4.5 Dataset Preparation 19

4.6 Data Preprocessing 20

4.7 Dataset Analysis 21

4.8 Model Design 22

4.8.1 Traditional Machine Learning Models: 22

4.8.2 Transformer-Based Models: 24

4.9 Ensemble Learning 25

4.10 Explainable AI 27

4.11 Evaluation Metrics 27

4.11.1 Accuracy, Precision, and Recall 27

4.11.2 Weighted F1 Score 27

4.11.3 Matthews Correlation Coefficient (MCC) 27

4.11.4 Area Under the ROC Curve (AUC) 28

5 RESULTS AND DISCUSSION **29**

5.1 Machine Learning Models 29

5.2 Transformer-Based Models 29

5.2.1 Overall Performance Comparison 30

5.2.2 XLM-Roberta (Base) 30
5.2.3 Multilingual MiniLM-L12-H384 31
5.2.4 NepBERTa 32
5.3 Ensemble Model Results 33
5.4 Explainability Analysis via SHAP 34
5.5 Key Findings 35
5.6 Limitations 36
5.7 DISCUSSION 36
6 CONCLUSION AND FUTURE ENHANCEMENTS 38
6.1 Conclusion 38
6.2 Future Enhancements 38
REFERENCES 40
APPENDIX A 42
APPENDIX B 44

LIST OF FIGURES

Figure 4.1: Block diagram of training pipeline of the proposed system 19

Figure 4.2: Dataset Preparation 20

Figure 4.3: Length distribution of human-written and AI-generated texts. . . . 22

Figure 4.4: Bert encoder with classification head 25

Figure 4.5: Ensemble Learning Architecture 26

Figure 5.1: Training and validation curves for XLM-Roberta (Base) 31

Figure 5.2: Confusion Matrix for XLM-Roberta (Base) 31

Figure 5.3: Training and validation curves for Multilingual MiniLM-L12-H384 32

Figure 5.4: Confusion Matrix for Multilingual MiniLM-L12-H384 32

Figure 5.5: Training and validation curves for NepBERTa 33

Figure 5.6: Confusion Matrix for NepBERTa 33

Figure 5.7: SHAP token-level contributions. Red highlights indicate features contributing toward an *AI* classification, while Blue highlights indicate contributions toward a *Human* classification. Darker shades represent stronger contributions. 35

Figure 6.1: Authentication screens of the application. 42

Figure 6.2: Input Page 42

Figure 6.3: Results Page 43

Figure 6.4: History Page 43

Figure 6.5: Sample Dataset Snippet 44

Figure 6.6: Human Data Wordcloud 44

Figure 6.7: AI Data Wordcloud 44

LIST OF TABLES

Table 4.1: Statistical summary of the dataset 22

Table 4.2: Model Hyperparameters 23

Table 4.3: Hyper-Parameters for NepBerta,XLM-Roberta, and Multilingual-MiniLM 25

Table 5.1: Performance Metrics of Traditional Machine Learning Models . . 29

Table 5.2: Accuracy of Baseline Models at Different Token Lengths 30

Table 5.3: Model Performance Comparison for 512 token length 30

Table 5.4: Performance Comparison of Model Combinations 34

Table 5.5: Full metrics results of the ensemble learning algorithms applied . 34

LIST OF ABBREVIATIONS

Adam	: Adaptive Moment Estimation
AI	: Artificial Intelligence
AUC	: Area Under Curve
BERT	: Bidirectional Encoder Representations from Transformers
CatBoost	: Categorical Boosting
CNN	: Convolutional Neural Network
DL	: Deep Learning
DT	: Decision Tree
GPT	: Generative Pre-trained Transformer
LIME	: Local Interpretable Model-agnostic Explanations
LLAMA	: Large Language Model Meta AI
LLM	: Large Language Model
LR	: Learning Rate
LSTM	: Long Short-Term Memory
MAE	: Mean Absolute Error
ML	: Machine Learning
MSE	: Mean Squared Error
NLP	: Natural Language Processing
RF	: Random Forest
RMSE	: Root Mean Squared Error
RNN	: Recurrent Neural Network
ROC	: Receiver Operating Characteristic
RoBERTa	: Robustly Optimized BERT Pretraining Approach
SGD	: Stochastic Gradient Descent
SHAP	: SHapley Additive exPlanations
SVC	: Support Vector Classifier
SVM	: Support Vector Machine
TF-IDF	: Term Frequency–Inverse Document Frequency
XGBoost	: Extreme Gradient Boosting
XLM-R	: Cross-lingual Language Model-RoBERTa

CHAPTER 1

INTRODUCTION

1.1 Background

Large Language Models (LLMs) have rapidly advanced in their ability to perform tasks such as translation, text generation, proofreading, exam grading, and logical reasoning. They mark a major milestone in the evolution of Artificial Intelligence. The formidable capabilities of LLMs have raised concerns about distinguishing between their generated texts and human-written content. These concerns stem from two main issues. Firstly, LLMs are prone to producing fabricated or outdated information, potentially leading to the spread of misinformation and plagiarism. Secondly, there is a risk of malicious use, including disinformation dissemination, online fraud, social media spam, and academic dishonesty by students using LLMs for essay writing. As LLMs increasingly mimic human writing, distinguishing AI-generated content from that written by humans has become a growing challenge, one that carries significant implications across multiple domains.

1.2 Gap Identification

Although progress has been made to detect AI-generated content in high-resource languages, there is a clear lack of research and tools focused on low-resource languages like Nepali. Furthermore, there is a notable lack of publicly available labeled datasets to distinguish between human-written and AI-generated Nepali text.

1.3 Motivation

With the growing use of Large Language Models (LLMs) in generating Nepali text, it is increasingly important to ensure that AI-generated content can be reliably detected to preserve academic integrity, journalistic authenticity, and public trust. The lack of effective detection tools for the Nepali language poses a risk of misinformation, plagiarism, and other forms of content misuse. This highlights the urgent need for a language-specific, transparent, and user-trustworthy detection system for Nepali con-

tent. By developing a robust model to detect AI-generated text, this study provides a valuable tool for educators, promoting fairness and academic integrity. The broader educational relevance lies in its application to plagiarism detection, the evaluation of digital submissions, and the safeguarding of learning outcomes in environments where AI-generated content is increasingly utilized.

1.4 Objectives

- To construct a labeled dataset of human-written and AI-generated Nepali text, addressing the scarcity of such resources.
- To conduct comprehensive experiments with traditional machine learning models and state-of-the-art language models on a text dataset, integrating explainable AI techniques to ensure transparency.

CHAPTER 2

RELATED THEORY

2.1 Natural Language Processing

Natural language processing (NLP) is a subfield of computer science and artificial intelligence (AI) that uses machine learning to enable computers to understand, process and generate human language. NLP enables computers and digital devices to recognize, understand and generate text and speech by combining computational linguistics, the rule-based modeling of human language, together with statistical modeling, machine learning and deep learning.

NLP is already part of everyday life for many, powering search engines, prompting chatbots for customer service with spoken commands, voice-operated GPS systems and question-answering digital assistants on smartphones such as Amazon's Alexa, Apple's Siri and Microsoft's Cortana.

2.2 Transformer

A Transformer is a deep learning architecture introduced in *"Attention Is All You Need"* [1]. It is specifically designed to handle sequential data, such as natural language text, using a mechanism called self-attention.

Unlike earlier models such as recurrent neural networks (RNNs) [2]and long short-term memory (LSTM) [3]networks, Transformers do not process data sequentially. Instead, they analyze the entire input sequence simultaneously. The self-attention mechanism allows the model to focus on different parts of the input when processing each token, which enables it to capture long-range dependencies more effectively.

Transformers have become the foundation of modern Natural Language Processing (NLP) systems and are widely used in tasks such as language modeling, machine translation, text classification, and question answering.

2.3 Language Models

A language model is an artificial intelligence system designed to understand, interpret, and generate human language. It learns patterns, structure, and context from large amounts of text data, enabling it to predict and produce meaningful text.

Language models play a crucial role in many applications, including speech recognition, machine translation, text generation, chatbots, search engines, and other NLP systems. By improving the ability of machines to process language, they enhance human-computer interaction.

Some well-known language models include:

- **GPT**[4] (Generative Pre-trained Transformer), developed by OpenAI, widely used for text generation and conversational AI.
- **BERT** [5](Bidirectional Encoder Representations from Transformers), developed by Google, designed to understand context in both directions within a sentence.
- **XLM-RoBERTa** [6](Cross-lingual RoBERTa), developed by Facebook AI, , an improved version of BERT with better training strategies, designed for multilingual and cross-lingual understanding.
- **MuRIL** [7](Multilingual Representations for Indian Languages), developed by Google Research, specifically tailored for Indic languages.
- **LLaMA** [8](Large Language Model Meta AI), developed by Meta, focused on efficient and research-oriented language modeling.
- **NepBERTa**[9], a transformer-based model developed for the Nepali language, aimed at improving NLP tasks for Nepali text.

2.3.1 Foundation Models

Foundation models are a class of large-scale artificial intelligence models that are pre-trained on vast amounts of diverse data and designed to serve as a general-purpose base for a wide variety of tasks. Rather than creating and training a new system from scratch for one narrow purpose, as older models did, foundation models learn broad patterns,

representations, and structures from the data. That design makes it possible to adapt them to many follow-up applications with only a little extra work.

At heart, the approach comes down to pre-training followed by adaptation. During pre-training, the model works through massive datasets of text, images, audio, or mixtures of all three, without being locked into any single goal. This exposure builds a solid grasp of the underlying patterns and connections in the information. Once that stage finishes, the model can be fine-tuned or simply prompted to manage specific jobs such as classification, translation, summarization, or question answering.

The remarkable adaptability of foundation models is a key feature. They demonstrate a capacity to generalize their knowledge, applying it to a variety of tasks and domains. This significantly reduces the necessity of constructing distinct systems for each new application. Consequently, both time and resources are utilized more efficiently. Furthermore, these models facilitate transfer learning, allowing the lessons learned in one context to be effectively leveraged in new and different scenarios.

2.3.2 The Fine-Tuning Process

Fine-tuning is the crucial step that turns a general-purpose model into a specialized expert ready for real-world use. While a base model grasps the fundamentals of language, fine-tuning supplies the targeted knowledge required for specific domains such as medicine, law, or technical support. This shift happens through a clear sequence of stages:

Phase 1: Pre-training

Before any specialization begins, the model completes Pre-training on a massive, multi-domain corpus. During this stage, it builds a broad understanding of grammar, logic, and general world facts.

Phase 2: Targeted Domain Exposure

Once the foundation is in place, the model receives a smaller, high-quality dataset focused on the particular task. This Fine-Tuning Data is usually labeled to steer the model

toward the desired behavior.

Phase 3: Parameter Optimization

In this stage, the model's internal weights receive small adjustments across a few training epochs. These changes help it emphasize the patterns in the new data while keeping the broad knowledge it gained during pre-training.

Phase 4: Evaluation and Validation

After tuning is complete, the model is tested on a separate validation set. Its performance is measured against standard metrics to make sure it delivers reliable and contextually accurate results before deployment.

2.4 Machine Learning Classifiers

Machine learning classifiers are supervised learning algorithms designed to map input features to discrete class labels by optimizing an objective function over a labeled training dataset. In the context of Natural Language Processing (NLP), these models operate on numerical vector representations of text. They remain staple baselines due to their computational efficiency, lower data requirements compared to deep learning, and inherent interpretability. The following subsections detail the mathematical foundations of the classifiers employed in this study.

2.4.1 Support Vector Classifier (SVC)

The Support Vector Classifier (SVC) is a supervised learning algorithm derived from the Support Vector Machine (SVM) framework, specifically designed for classification tasks. It constructs an optimal separating hyperplane between classes in a high-dimensional feature space. The primary objective of SVC is to maximize the margin—the geometric distance between the hyperplane and the nearest training data points, known as support vectors.

This margin-maximization property enhances the model's generalization capabilities, particularly in high-dimensional settings such as text classification, where the number

of features often exceeds the number of samples. For a binary classification task with training pairs $\{(x_i, y_i)\}$, where $y_i \in \{-1, +1\}$, SVC solves the following optimization problem:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w} \cdot x_i + b) \geq 1, \quad \forall i \tag{2.1}$$

In practical implementations, such as in the SVC module of machine learning libraries, regularization is controlled via a parameter C, which balances margin maximization and classification error. Additionally, kernel functions (e.g., linear, polynomial, radial basis function) can be applied to handle non-linear decision boundaries.

2.4.2 Logistic Regression

Logistic Regression is a fundamental probabilistic linear classifier. Despite its name, it is used for classification by applying the sigmoid function to a linear combination of input features to produce a posterior probability estimate:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot x + b)}} \tag{2.2}$$

The model parameters are optimized by minimizing the cross-entropy (log loss) over the training set. Logistic Regression is computationally efficient and naturally interpretable, as its coefficients provide direct insight into the influence of specific features on the classification outcome.

2.4.3 Naive Bayes

Naive Bayes is a generative probabilistic classifier grounded in Bayes' Theorem. It calculates the posterior probability of a class C given an observation vector x as:

$$P(C \mid x) \propto P(C) \prod_{i=1}^{n} P(x_i \mid C) \tag{2.3}$$

The "naive" assumption posits that all features are conditionally independent given the class label. While this assumption is rarely strictly met in natural language, Naive

Bayes remains a powerful baseline for text classification due to its speed and its ability to perform well even with limited training data.

2.4.4 Random Forest

Random Forest is an ensemble learning method that utilizes bagging to construct a multitude of decision trees. During training, each tree is built using a random subset of the training data and a random selection of features at each split. The final prediction is determined via majority voting:

$$\hat{y} = \arg\max_{c} \sum_{t=1}^{T} \mathbb{I}(\hat{y}_t = c) \tag{2.4}$$

where T is the total number of trees and $\mathbb{I}$ is the indicator function for the prediction of the t-th tree. By aggregating the predictions of decorrelated trees, Random Forest significantly reduces variance without increasing bias, resulting in a robust and generalizable classifier.

2.4.5 XGBoost

XGBoost (Extreme Gradient Boosting) is a highly optimized sequential ensemble method based on gradient boosting. Unlike bagging methods, XGBoost builds trees iteratively, where each successive tree $h_m(x)$ is trained to minimize the residual errors of the preceding ensemble:

$$\mathscr{L}^{(m)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(m-1)} + h_m(x_i)\right) + \Omega(h_m) \tag{2.5}$$

where l is a differentiable loss function and $\Omega(h_m)$ is a regularization term that penalizes model complexity. XGBoost incorporates advanced optimizations, such as column subsampling and efficient handling of sparse data, making it exceptionally effective for complex structured data tasks.

2.5 Tokenization

Tokenization is the fundamental process of segmenting a sequence of text into smaller, discrete units called tokens. These units act as the building blocks for all subsequent NLP tasks.

2.5.1 Strategies for Tokenization

- **Word-Level Tokenization:** Text is split based on whitespace and punctuation. While intuitive, it struggle with large vocabularies and "Out-of-Vocabulary" (OOV) words.
  - *Input:* "AI is evolving." → *Tokens:* ["AI", "is", "evolving", "."]
- **Subword Tokenization:** This approach breaks words into meaningful fragments (prefixes, roots, suffixes). It is the standard for modern LLMs because it handles rare words and complex morphology efficiently.
  - *Input:* "dissatisfaction" → *Tokens:* ["dis", "satisfact", "ion"]
- **Character-Level Tokenization:** Text is sliced into individual characters. This is useful for languages with no clear word boundaries or for models that need to be robust against misspellings.
  - *Input:* "computer" → *Tokens:* ["c", "o", "m", "p", "u", "t", "e", "r"]

2.6 Embeddings and Vectorization

To process text, machines require a numerical representation. This is achieved through Embeddings and Vectorization.

2.6.1 Embeddings

Embeddings are dense vector representations where tokens with similar meanings are mapped to nearby points in a multi-dimensional mathematical space. Algorithms such as Word2Vec, GloVe, and Transformer-based embeddings encode semantic context into these vectors. For instance, the vector for "cat" will share more mathematical properties

with ”dog” than with ”car.”

2.6.2 Vectorization

Vectorization is the overarching process of converting raw data into a numerical format. While embeddings handle individual tokens, vectorization techniques like TF-IDF or Sentence Transformers allow entire documents to be represented as arrays. This mathematical bridge is essential for performing the complex calculations required by Large Language Models.

2.7 Optimization and Loss Functions

Training a deep learning model involves navigating a complex, multidimensional landscape of parameters to find the weights that minimize prediction error.

2.7.1 Cross-Entropy Loss

Cross-entropy loss is the standard objective function for classification tasks. It measures the divergence between the ground-truth probability distribution (y) and the model's predicted probabilities ($\hat{y}$). By heavily penalizing confident but incorrect predictions, it encourages the model to align its outputs with the true class labels.

$$\text{Cross-Entropy Loss} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i) \tag{2.6}$$

In this work, *label smoothing* is applied to improve generalization and prevent the model from becoming overconfident. Instead of using a one-hot encoded target distribution, the true labels are smoothed by assigning a small probability to all non-target classes. This regularization technique helps reduce overfitting and makes the model more robust.

The smoothed target distribution is defined as:

$$y_i^{LS} = \begin{cases} 1-\varepsilon & \text{if } i = y \\ \frac{\varepsilon}{K-1} & \text{otherwise} \end{cases} \tag{2.7}$$

where ε is the smoothing factor and K is the number of classes.

2.7.2 Adam Optimizer

The **Adam (Adaptive Moment Estimation)** optimizer updates model weights by combining two powerful techniques:

- **Momentum:** Accelerates gradient descent in the correct direction by tracking the ”first moment” (mean of past gradients).
- **RMSProp:** Adapts the learning rate for each parameter by tracking the ”second moment” (variance of past gradients).

2.8 Explainable AI (XAI)

As models grow in complexity, understanding the logic behind a prediction is critical for building trust and ensuring transparency.

2.8.1 SHAP (SHapley Additive exPlanations)

Based on cooperative game theory, SHAP[10] provides a mathematically grounded way to distribute ”credit” for a prediction among input features. It calculates Shapley values to determine how much each feature (e.g., a specific word in a sentence) contributed to the final output.

2.9 Evaluation Metrics

To quantify effectiveness, we rely on a set of metrics derived from the confusion matrix, which categorizes predictions into four types:

- **True Positive (TP):** Correctly predicted positive instances.
- **True Negative (TN):** Correctly predicted negative instances.
- **False Positive (FP):** Negative instances incorrectly flagged as positive (Type I error).
- **False Negative (FN):** Positive instances incorrectly flagged as negative (Type II

error).

2.9.1 Accuracy, Precision, and Recall

These metrics provide different perspectives on performance:

- **Accuracy:** The overall proportion of correct predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{2.8}$$

- **Precision:** The model's "exactness", out of all positive predictions, how many were correct?

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2.9}$$

- **Recall (Sensitivity):** The model's "completeness", out of all actual positive cases, how many did the model find?

$$\text{Recall} = \frac{TP}{TP + FN} \tag{2.10}$$

2.9.2 Macro-F1 Score

In imbalanced datasets, simple accuracy can be misleading. The Macro-F1 Score is the harmonic mean of precision and recall, averaged across all N classes. It treats every class equally, providing a robust measure of performance regardless of class size.

$$\text{Macro-F1} = \frac{1}{N} \sum_{i=1}^{N} \frac{2 \times P_i \times R_i}{P_i + R_i} \tag{2.11}$$

2.9.3 Weighted F1 Score

The Weighted F1 Score computes the F1-score for each class and averages them, weighting by the number of true instances (support) in each class:

$$\text{Weighted-F1} = \frac{\sum_{i=1}^{N} \text{support}_i \times F1_i}{\sum_{i=1}^{N} \text{support}_i} = \frac{\sum_{i=1}^{N} \text{support}_i \times \frac{2P_iR_i}{P_i+R_i}}{\sum_{i=1}^{N} \text{support}_i} \tag{2.12}$$

where P_i and R_i are the precision and recall for class i, and support_i is the number of true instances of class i.

2.9.4 Matthews Correlation Coefficient (MCC)

The Matthews Correlation Coefficient is a highly robust metric that provides a balanced measure of quality, even in the presence of significant class imbalance. Unlike the F1-score, MCC utilizes all four quadrants of the confusion matrix. Its value ranges from -1 (total disagreement) to $+1$ (perfect prediction), where 0 indicates a performance no better than random guessing.

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \tag{2.13}$$

2.9.5 Area Under the ROC Curve (AUC)

The Receiver Operating Characteristic (ROC)curve is a graphical plot that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied. It plots the True Positive Rate (TPR or Sensitivity) against the False Positive Rate (FPR).

The Area Under the Curve(AUC) quantifies the overall ability of the model to discriminate between positive and negative classes. An AUC of 1.0 represents a perfect model, while an AUC of 0.5 suggests the model has no class separation capacity whatsoever.

$$TPR = \frac{TP}{TP+FN} \tag{2.14}$$

$$FPR = \frac{FP}{FP+TN} \tag{2.15}$$

2.10 Ensemble Learning Architectures

Ensemble learning leverages the collective intelligence of multiple models to achieve superior predictive accuracy. The experiments conducted in this study utilize three fundamental approaches: Parallel (Bagging), Sequential (Boosting), and Hierarchical (Stacking).

2.10.1 Voting Mechanisms

Voting ensembles serve as a baseline for model aggregation.

- **Soft Voting:** The final class $\hat{y}$ is determined by averaging the probability distributions P of n classifiers:

$$\hat{y} = \arg\max_i \sum_{j=1}^{n} w_j P_{ij} \tag{2.16}$$

- **Hard Voting:** The final class $\hat{y}$ is determined by majority voting among the predictions of n classifiers. Each classifier votes for a class label, and the class with the highest number of votes is selected:

$$\hat{y} = \arg\max_i \sum_{j=1}^{n} w_j \cdot 1(\hat{y}_{=} i) \tag{2.17}$$

  where $\hat{y}_j$ is the predicted class of the j-th classifier, w_j is the weight assigned to that classifier, and $1\ (\cdot)$ is an indicator function that equals 1 if the condition is true and 0 otherwise.

- **Weighted Voting:** Assigns relative importance w to each base learner based on their cross-validation performance.

2.10.2 Bagging and Boosting

- **Bagging:** Focuses on reducing variance by training base learners on "bootstrapped" subsets of the data. The final output is an average of these independent models.
- **Boosting:** Reduces bias and variance sequentially. Algorithms like XGBoost and CatBoost iteratively add trees that minimize the residual loss of the preceding ensemble.

2.10.3 Stacked Generalization

Stacking introduces a two-tier architecture. The first layer consists of diverse base learners (Random Forest, SVC, etc.), while the second layer is a Meta-Learner. The

meta-learner is trained to recognize which base models are most reliable for specific types of data points, effectively learning the ”optimal combination” of the underlying models.

CHAPTER 3

LITERATURE REVIEW

The rapid evolution of Large Language Models (LLMs) has enabled machines to generate human-like text, blurring the line between content written by humans and machines. As a result, there is a growing need to develop effective techniques for detecting AI-generated text. Machine-generated text detection has evolved from traditional machine learning methods to more advanced and specialized approaches. Early techniques relied on models such as Naive Bayes, Logistic Regression, Support Vector Machine, and Random Forest, using handcrafted features like n-grams[11], TF-IDF, and stylometric patterns to differentiate human and machine-written text. However, these approaches struggle with modern, highly fluent AI-generated content. In addition, watermark-based detection[12] introduces hidden statistical patterns during text generation, allowing direct identification of AI-generated content. While effective, this method depends on access to the generation model and is vulnerable to modifications such as paraphrasing or translation.

More recently, large pretrained language models like DeBERTa [13], RoBERTa [6], and T5 [14] have been fine-tuned for AI text detection task. These models are typically fine-tuned on labeled datasets containing both human-written and AI-generated text which relies solely on the quality of text generated by AI model. However, they struggle when newer language model which is trained on large corpus is used to generate text [15]. Several AI text detection tools, including GPTZero, Originality.ai, Copyleaks AI Content Detector, and DetectGPT have been developed which are trained on high-resource languages like English. [16] applied classical machine learning techniques such as Gradient Boosting, Stacking, and Voting. Instead of using raw text as features, these models used probability outputs from various models as input features.

Most detection models have been developed and benchmarked in high-resource languages such as English and perform poorly on low-resource languages, leaving a significant research gap in low-resource languages like Nepali. The scarcity of Nepali datasets for generative or classification tasks presents a key bottleneck in advancing NLP research in the region. As detection systems become more complex, there is

an increasing need for transparency and interpretability. Explainable AI (XAI) techniques such as LIME [17] and SHAP [10] offer local explanations for predictions by approximating the contribution of each input feature. In the context of AI-generated text detection, these tools can help users understand why a text is classified as AI or human-written, which is particularly important in domains like education, media, and law.

While advancements in LLM detection are promising, low-resource languages such as Nepali remain underserved. This project aims to fill this gap by curating a labeled Nepali dataset, developing AI-detection models tailored for the language, and incorporating explainability features to enhance trust and transparency.

CHAPTER 4

METHODOLOGY

4.1 Overview

This methodology outlines the proposed approach to design, develop, and evaluate a system capable of distinguishing between AI-generated and human-authored Nepali text. The project focuses on creating a high-quality Nepali dataset, developing classification models, and integrating explainable AI techniques to ensure interpretability and transparency of model predictions.

4.2 Software Development Model

The proposed system was developed using an Agile software development model. Agile model was chosen because it is particularly well-suited for research-driven and experimental work, where specific requirements often shift as new data or technical insights emerge during the building process.

Rather than a single, rigid production phase, the project was broken down into short development cycles or sprints. Each of these sprints targeted a distinct pillar of the architecture, such as data gathering, text preprocessing, model training, or the integration of explainability tools. This approach ensured that a functional piece of the system was produced and analyzed at the end of every cycle.

Ultimately, adopting an Agile mindset provided the balance we needed: a structured, professional workflow that was still nimble enough to evolve alongside our experimental findings.

4.3 Research and Problem Analysis

The initial phase involved a detailed literature review of existing methods used for detecting AI-generated text. We analyzed the limitations of current approaches in high-resource languages and identified key challenges when applying these methods to Nepali, a low-resource linguistic environment. This analysis helped define the tech-

nical requirements and select appropriate models tailored for the Devanagari script.

4.4 Training Pipeline

The architecture of the proposed system is shown in the diagram. It presents a pipeline designed to classify Nepali text as either AI-generated or human-written.

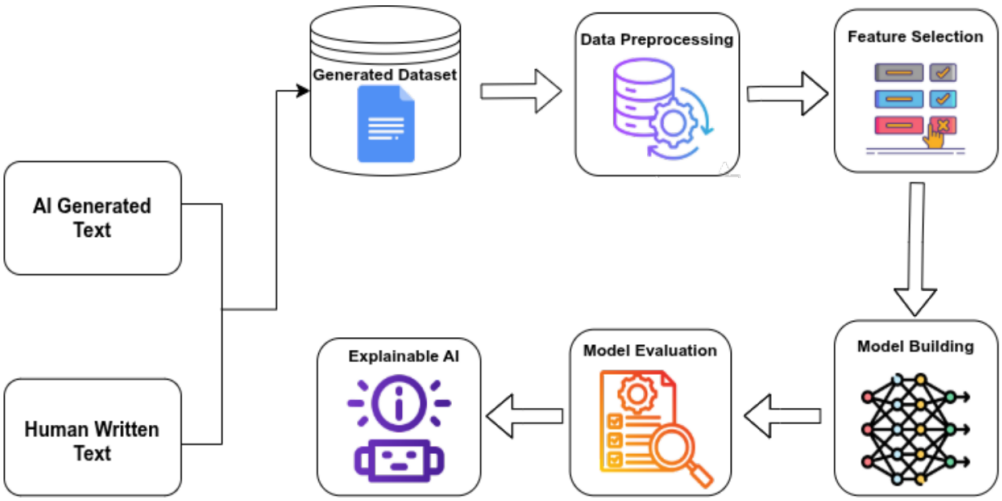


Figure 4.1: Block diagram of training pipeline of the proposed system

The system takes AI-generated and human-written text as input to form a combined dataset. This data is then preprocessed to remove noise and standardize the text. Relevant features are extracted and used to train classification models.

The trained models are evaluated using standard performance metrics to assess their effectiveness. Finally, explainable AI techniques are applied to interpret the model's decisions, improving transparency and understanding of the results.

4.5 Dataset Preparation

Currently, in Nepal , there is no dataset that is specifically created to serve the task of AI-generated Nepali text detection . Therefore, we developed a custom dataset strategy. The dataset was prepared carefully and with high quality to ensure that it meets the requirements of AI text detection task. As the task is to distinguish between human-written and AI-generated text, we employed different data collection strategies for each label, as described below.

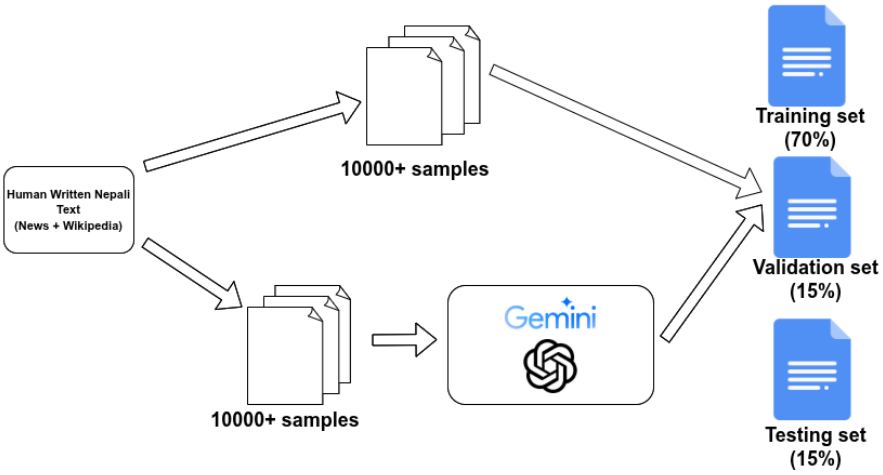


Figure 4.2: Dataset Preparation

- **Human-written Text:** Sourced from a publicly available Nepali news dataset on HuggingFace, containing authentic articles from *Kantipur*, *Gorkhapatra*, and *BBC Nepali* and wikipedia.
- **AI-Generated Text:** Generated from human written text samples using commonly used text generation models, **Google Gemini 2.5**, **Google Gemini 3.1 flash lite** and **gpt-5-mini**. We utilized diverse prompt strategies including *Rewrite*, *Polish*, *Complete*, and *Generate from Title* to capture variance in writing style and structuring, thereby enhancing the robustness of the dataset.

4.6 Data Preprocessing

The raw Nepali text data consisting of both human-written and AI-generated samples, underwent several preprocessing steps to prepare it for effective model training. The preprocessing pipeline is summarized as follows:

- **Unicode Normalization:** Unicode normalization was applied to ensure consistent encoding of Nepali characters, particularly to correctly handle combined vowels and diacritics.
- **Removal of Numbers:** All numerical digits, including both Nepali and English digits (0–9), were removed to eliminate noise introduced by numerical data.
- **Removal of Punctuation and Special Characters:** All punctuation marks and special symbols were stripped from the text to retain only meaningful linguistic

content.

- **Tokenization:** A Unicode-aware tokenizer designed for the Devanagari script was used to split text into meaningful Nepali tokens without breaking grapheme clusters.
- **Stopword Removal:** Nepali stopwords were removed to reduce noise and eliminate high-frequency function words that carry limited semantic information, allowing the classifiers to focus on more informative content words.
- **Whitespace Normalization:** Extra spaces and line breaks were removed to standardize text formatting across the dataset.

These preprocessing steps helped improve overall data quality and ensured that the downstream classification models learned from clean and meaningful Nepali text features. For model development and evaluation, the dataset was split into training and testing sets using an 80:20 ratio.

4.7 Dataset Analysis

We analyze the dataset by comparing the length distributions of human-written and AI-generated texts, as summarized in Table 4.1. The two classes contain a equal number of samples indicating that the dataset is balanced. On average, AI-generated texts are shorter than human-written texts, indicating that AI models tend to produce shorter texts. The median length of human-written text is slightly higher than that of AI-generated texts, while the average length is significantly higher. This indicates that human written texts have some very long texts that increase the average length, while AI-generated texts are more consistent, as shown by the smaller difference in the median length and average length. These differences in length characteristics highlight structural distinctions between human and AI-generated writing that may be informative for classification tasks.

Table 4.1: Statistical summary of the dataset

Metric	Human Written	AI Generated
Size	12873	12873
Avg. Length	371	291
Median Length	263	249
Max Length	2441	2085
Min Length	100	100

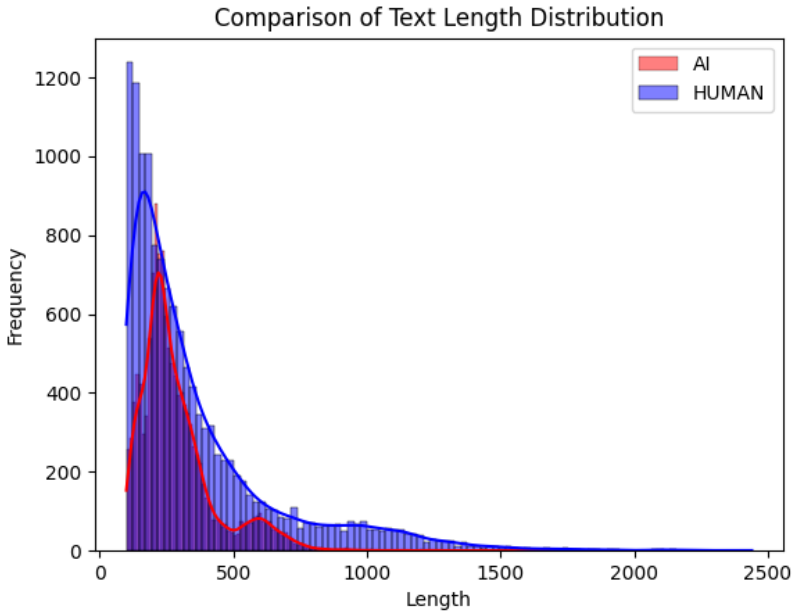


Figure 4.3: Length distribution of human-written and AI-generated texts.

4.8 Model Design

To identify AI-generated text in Nepali, we experimented with both traditional machine learning models and transformer-based deep learning models.

4.8.1 Traditional Machine Learning Models:

After preprocessing and vectorizing the dataset, we trained and evaluated five baseline classifiers. The following traditional machine learning methods were used for the LLM detection task:

1. Naive Bayes
2. Logistic Regression

3. Random Forest

4. XGBoost

5. Support Vector Machines

Table 4.2: Model Hyperparameters

Model	**Hyperparameters**
SVC (Linear + Calibration)	C = 1.0 max_iter = 500 random_state = 42 calibration = Platt scaling (CalibratedClassifierCV)
Multinomial Naive Bayes	alpha = 0.1
Logistic Regression	C = 1.0 max_iter = 500 solver = lbfgs random_state = 42
Random Forest	n_estimators = 100 max_depth = 4 n_jobs = -1 random_state = 42
XGBoost	n_estimators = 500 learning_rate = 0.05 max_depth = 6 subsample = 0.8 colsample_bytree = 0.8 eval_metric = logloss tree_method = hist n_jobs = -1

These models were selected to represent a diverse range of traditional approaches. Naive Bayes and Logistic Regression provide baseline performance for probabilistic and linear models, respectively. Naive Bayes is particularly effective for text classification tasks due to its simplicity and efficiency in handling high-dimensional data. Logistic Regression, on the other hand, is valued for its interpretability and ability to model binary outcomes

Random Forest and XGBoost represent ensemble methods known for their robustness and strong performance across various tasks. Random Forest utilizes multiple decision trees to improve predictive accuracy and reduce overfitting through averaging. XGBoost, a gradient boosting framework, is widely recognized for its computational efficiency and high performance, often outperforming other algorithms in structured data

scenarios. Support Vector Classification (SVC) was utilized with a linear formulation. Linear SVC offers computational efficiency and strong generalization in classification tasks. Since LinearSVC does not inherently provide probabilistic outputs, we applied Platt scaling via CalibratedClassifierCV, enabling probability estimation

For feature extraction, we used a TF-IDF (Term Frequency–Inverse Document Frequency) vectorizer that uses character-level n-grams instead of whole words, making it especially effective for languages like Nepali written in Devanagari. It extracts character sequences of length 2 to 4 to capture subword patterns, keeps only the top 20,000 most relevant features to stay efficient, applies logarithmic scaling to reduce the impact of very frequent patterns, and ignores rare ones that appear in fewer than two documents. Overall, it provides a compact and robust way to represent text, especially when dealing with spelling variations or noisy data.

```
vec = TfidfVectorizer(
    analyzer='char_wb',
    ngram_range=(2, 4),
    max_features=20_000,
    sublinear_tf=True,
    min_df=2
)
```

4.8.2 Transformer-Based Models:

In addition to traditional machine learning approaches, we finetuned and evaluated transformer-based models for our task. The following transformer models were used for classification:

1. XLM-RoBERTa(base)
2. NepBERTa
3. Multilingual-MiniLM-L12-H384(MiniLM)

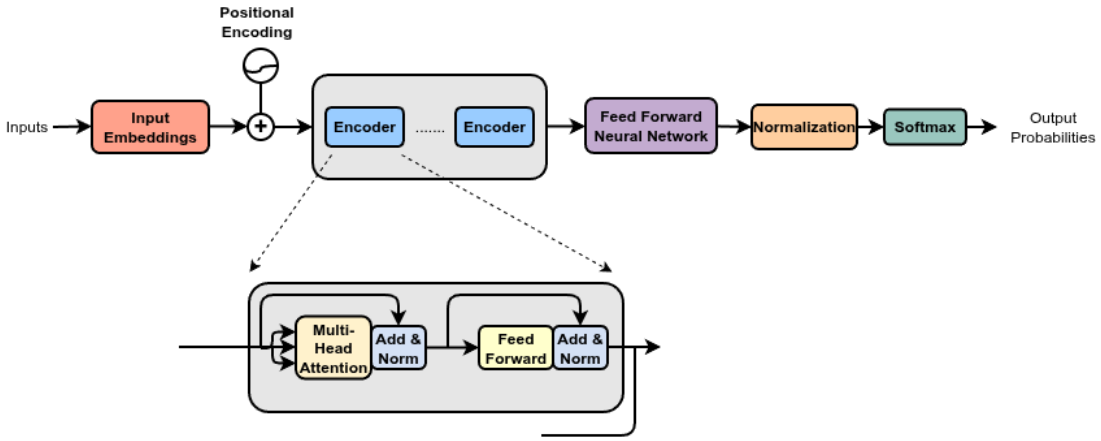


Figure 4.4: Bert encoder with classification head

Table 4.3: Hyper-Parameters for NepBerta,XLM-Roberta, and Multilingual-MiniLM

Hyper-Parameters	**NepBERTa**	**XLM-Roberta**	**MiniLM**
Epoch	10	10	10
Batch Size	16	16	16
Learning Rate	2×10^{-5}	2×10^{-5}	2×10^{-5}
Weight Decay	0.01	0.01	0.01
Optimizer	AdamW	AdamW	AdamW
Label Smoothing	0.1	0.1	0.1
Loss Function	Cross entropy	Cross entropy	Cross entropy
Scheduler	Linear (10% warmup)	Linear (10% warmup)	Linear (10% warmup)

4.9 Ensemble Learning

To enhance the performance of AI-generated text detection, five different ensemble techniques were applied. These methods include Voting, Stacking,Bagging and Gradient Boosting classifiers. All models were trained using class probability scores produced by base models for the validation dataset. Out of the five machine learning models and three transformer-based models, combinations were selected based on those that produced the best performance. The ensemble approaches used are described below:

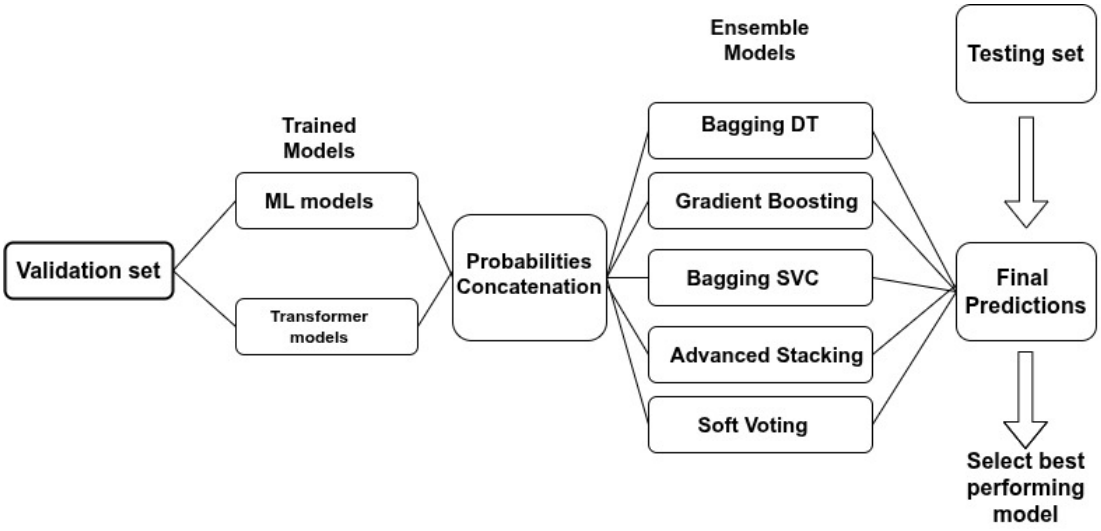


Figure 4.5: Ensemble Learning Architecture

1. **Bagging with Decision Tree** This method applies bootstrap aggregating (bagging) using Decision Trees as base learners. Multiple training subsets are created through sampling with replacement, allowing each tree to learn from slightly different data. This helps in reducing variance and improving model stability.

2. **Bagging with SVC** In this technique, several Support Vector Classifier (SVC) models are trained on different bootstrap samples. The predictions from all models are then combined using majority voting, which helps reduce variance while preserving the strong decision boundaries of SVC.

3. **Soft Voting** Soft voting combines predictions by averaging the probability outputs of multiple models. The final class is chosen based on the highest average probability across all models.

4. **Gradient Boosting Classifier** This method builds models sequentially, where each new model focuses on correcting the mistakes of the previous ones. It iteratively learns from residual errors, gradually improving overall performance by giving more attention to misclassified samples.

5. **Stacking (with Gradient Boosting as Meta-Learner)** Stacking follows a two-level structure. In the first level, multiple diverse models Random Forest, XGBoost, LightGBM, and CatBoost are trained on the dataset. In the second level, a Gradient Boosting Classifier acts as a meta-learner, using the predictions of the base models to produce the final output.

4.10 Explainable AI

To enhance model interpretability, SHAP (SHapley Additive exPlanations) is employed. SHAP is a game-theoretic approach that provides both local and global explanations by assigning importance values to input features based on their contribution to model predictions. This enables a deeper understanding of the decision-making behavior of both traditional machine learning models and transformer-based classifiers.

4.11 Evaluation Metrics

To assess model performance, we employed metrics suitable for multi-class and imbalanced datasets: accuracy, precision, recall, Weighted F1, Matthews Correlation Coefficient (MCC), and Area Under the ROC Curve (AUC). These metrics were computed using standard Python libraries.

4.11.1 Accuracy, Precision, and Recall

Accuracy measures the proportion of correct predictions. Precision evaluates the correctness of positive predictions, and recall (sensitivity) assesses the model's ability to identify positive instances. These metrics provide complementary views of model performance and are particularly informative for datasets with imbalanced class distributions.

4.11.2 Weighted F1 Score

The Weighted F1 Score combines precision and recall into a single measure for each class and averages them, weighting by the number of true instances (support) per class. This ensures that minority classes contribute appropriately to the overall evaluation.

4.11.3 Matthews Correlation Coefficient (MCC)

MCC provides a balanced measure of classification quality by considering all entries of the confusion matrix. It is particularly useful for imbalanced datasets, with values ranging from -1 (total disagreement) to $+1$ (perfect prediction), and 0 indicating random

performance.

4.11.4 Area Under the ROC Curve (AUC)

The ROC curve plots the True Positive Rate against the False Positive Rate across different decision thresholds, and the AUC quantifies the model's overall ability to discriminate between positive and negative classes. An AUC of 1.0 indicates perfect discrimination, while 0.5 is equivalent to random guessing. AUC is especially relevant when evaluating models across varying thresholds.

All metrics were computed using the default implementations in Python libraries, ensuring consistency with standard evaluation practices.

CHAPTER 5

RESULTS AND DISCUSSION

In this section, we present the experimental findings of our Research. we evaluated traditional ML models , transformer models and ensemble methods on our test dataset. First, we analyze the performance metrics of traditional ML models and transformer models and then we explore how combining these models through various ensemble models affect the performance metrics.

5.1 Machine Learning Models

In the analysis of various ML model on test dataset, it is observed that SVM with linear kernel and probability calibration achieved highest performance across all evaluation metrics. XGBoost closely followed with comparable results indicating both models are effective for the task. Logistic Regression also showed strong performance, while Random Forest and Naive Bayes showed relatively lower effectiveness.

Table 5.1: Performance Metrics of Traditional Machine Learning Models

Model Name	**Accuracy**	**F1-score**	**MCC**	**AUC-ROC**
SVC	**0.9616**	**0.9616**	**0.9254**	0.9924
XGBoost	0.9610	0.9610	0.9226	**0.9929**
Logistic Regression	0.9413	0.9412	0.8852	0.9892
Random Forest	0.8864	0.8860	0.7778	0.9605
Naive Bayes	0.8629	0.8620	0.7337	0.9334

5.2 Transformer-Based Models

Transformer models, utilizing deep learning and attention mechanisms, were fine-tuned on the Nepali dataset. These models demonstrate a more sophisticated understanding of Devanagari syntax and context.

Firstly, we calculated the average accuracy across all models and token lengths. The average accuracy is 0.9236 for 128 tokens , 0.9398 for 256 tokens and 0.9602 for 512 tokens. The experiment shows that increasing the number of tokens in-fact improves the performance in our task.

Table 5.2: Accuracy of Baseline Models at Different Token Lengths

Baseline Models	**Accuracy**		
	128 tokens	**256 tokens**	**512 tokens**
NepBERTa	0.9226	0.9391	0.9677
XLM-Roberta(base)	0.9295	0.9482	0.9728
Multilingual-MiniLM-L12-H384	0.9238	0.9462	0.9715
Avg	0.9253	0.9445	0.9706

5.2.1 Overall Performance Comparison

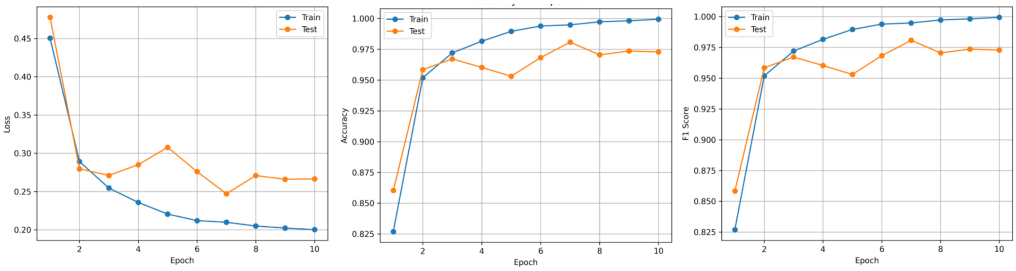
XLM-Roberta(base) performed best among all the models across all performance metrics. However, Multilingual-MiniLM-L12-H384 model achieved comparative performance despite having less parameters than XLM-Roberta(base). NepBERTa performed poorly among the models used for the study.

Table 5.3: Model Performance Comparison for 512 token length

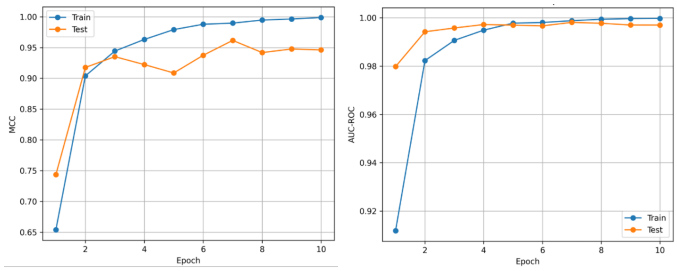
Model	**Params**	**Accuracy**	**F1-score**	**MCC**	**AUC-ROC**
XLM-Roberta(base)	270M	**0.9728**	**0.9728**	**0.9535**	**0.9965**
Multilingual-MiniLM-L12-H384	117M	0.9715	0.9715	0.9441	0.9950
NepBERTa	110M	0.9677	0.9677	0.9354	0.9942

5.2.2 XLM-Roberta (Base)

XLM-Roberta demonstrated competitive performance with slightly lower loss compared to NepBERTa.



(a) Loss (b) Accuracy (c) F1 Score



(d) MCC (e) AUC-ROC

Figure 5.1: Training and validation curves for XLM-Roberta (Base)

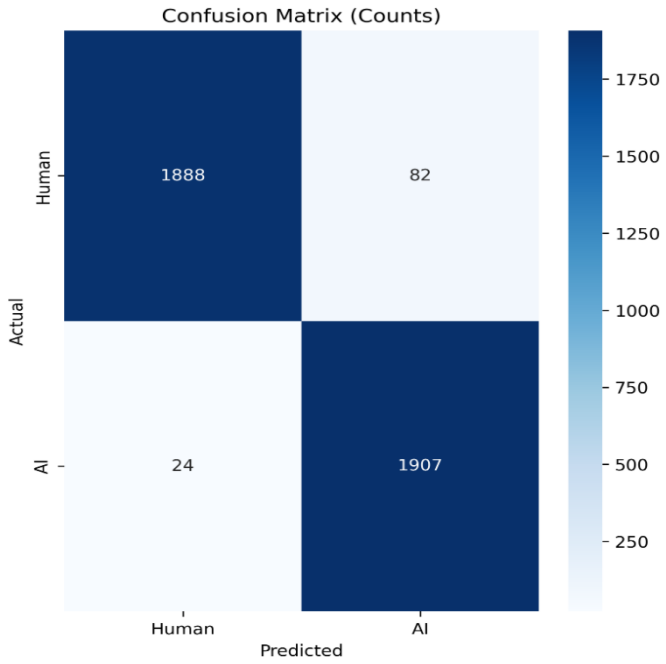
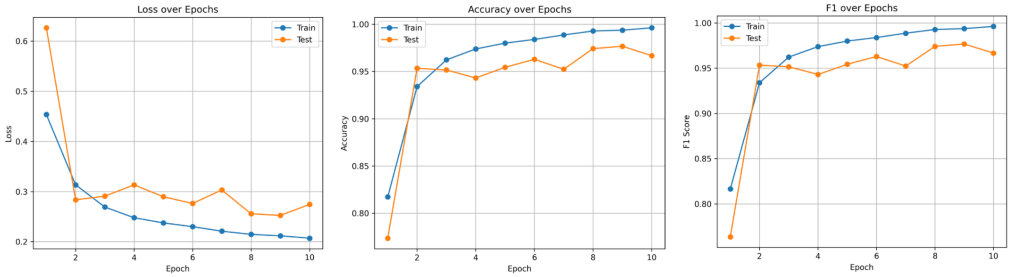


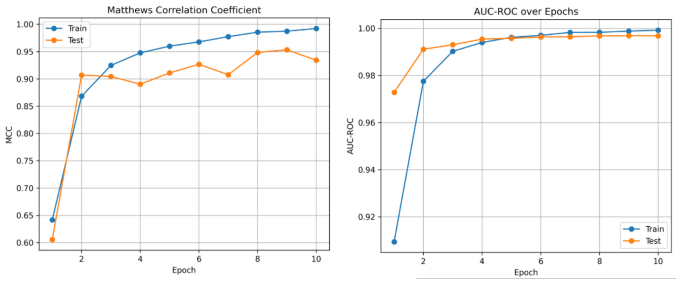
Figure 5.2: Confusion Matrix for XLM-Roberta (Base)

5.2.3 Multilingual MiniLM-L12-H384

Multilingual MiniLM achieved comparable accuracy while maintaining a smaller model size.



(a) Loss (b) Accuracy (c) F1 Score



(d) MCC (e) AUC-ROC

Figure 5.3: Training and validation curves for Multilingual MiniLM-L12-H384

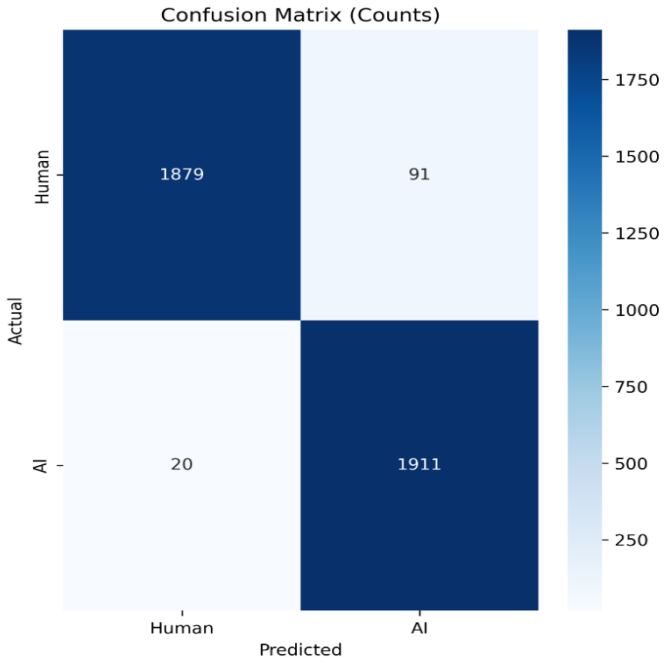
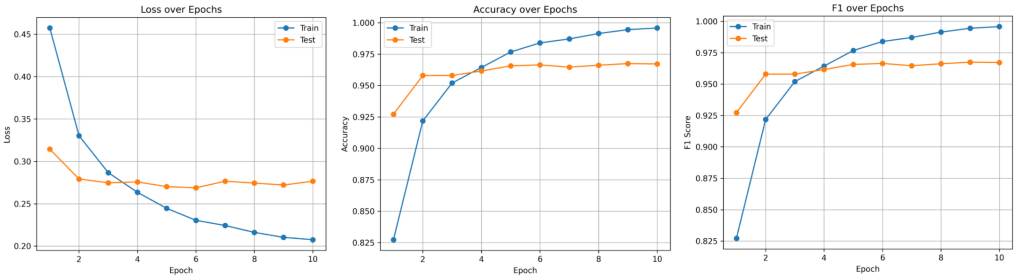


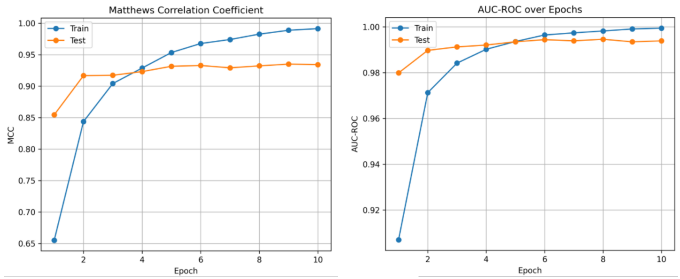
Figure 5.4: Confusion Matrix for Multilingual MiniLM-L12-H384

5.2.4 NepBERTa

NepBERTa was fine-tuned on the Nepali dataset and achieved strong performance across all evaluation metrics.



(a) Loss (b) Accuracy (c) F1 Score



(d) MCC (e) AUC-ROC

Figure 5.5: Training and validation curves for NepBERTa

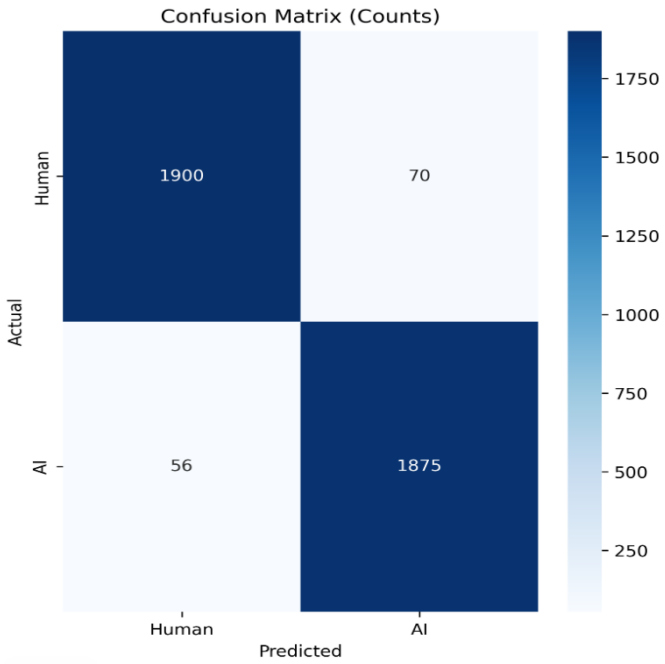


Figure 5.6: Confusion Matrix for NepBERTa

5.3 Ensemble Model Results

The combination of SVC, XLM-RoBERTa, NepBERTa and MiniLM was selected as the final model, as it demonstrated superior performance across all evaluation metrics compared to other combinations.

Table 5.4: Performance Comparison of Model Combinations

SVC	XG-Boost	RoBERTa	NepBERTa	MiniLM	Accuracy
✓	✓	✓	✓	✓	0.9841
✓	✓	×	✓	✓	0.9810
✓	✓	✓	×	✓	0.9848
✓	×	✓	✓	✓	**0.9853**
✓	✓	✓	✓	×	0.9846
×	✓	✓	✓	✓	0.9846
✓	✓	✓	×	×	0.9830
✓	✓	×	✓	×	0.9789
✓	✓	×	×	✓	0.9797
✓	×	✓	×	✓	0.9846
✓	×	✓	✓	×	0.9851
✓	×	×	✓	✓	0.9812
×	✓	×	✓	✓	0.9802
×	✓	✓	×	✓	0.9841
×	✓	✓	✓	×	0.9841
×	×	✓	✓	✓	0.9838
✓	×	✓	×	×	0.9815

Among all methods, The Advanced Stacking (Gradient Boosting Tree) model achieved the best results for the selected combination with accuracy and F1-score around 0.9853, demonstrating the effectiveness of combining multiple base learners. The Bagging Decision Tree ranked second with an accuracy of 0.9841, followed by the Gradient Boosting Classifier , soft voting and Bagging SVC with the lowest performance.

The results demonstrate that ensemble techniques significantly enhance model performance.

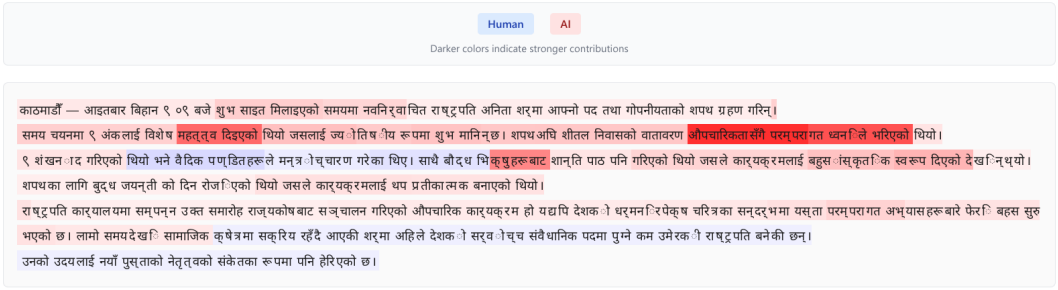
Table 5.5: Full metrics results of the ensemble learning algorithms applied

Model Name	**Accuracy**	**F1-score**	**MCC**	**AUC-ROC**
Advanced Stacking(GB Tree)	**0.9853**	**0.9853**	**0.9708**	**0.9984**
Bagging DT	0.9841	0.9841	0.9682	0.9971
Gradient Boosting Classifier	0.9833	0.9833	0.9666	0.9981
Soft Voting	0.9831	0.9831	0.9618	0.9977
Bagging SVC	0.9810	0.9810	0.9621	0.9983

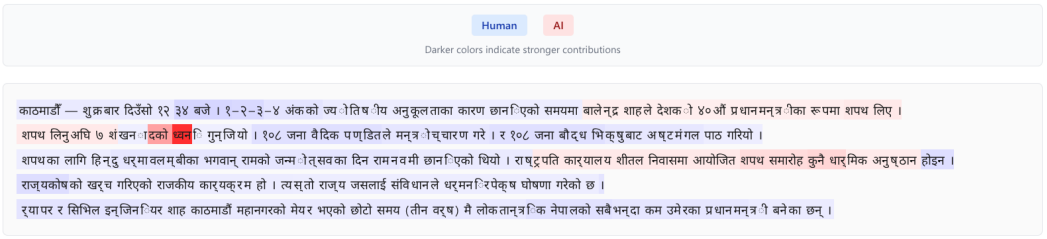
5.4 Explainability Analysis via SHAP

To enhance the interpretability of our classification system, we integrated SHAP (SHapley Additive exPlanations) to identify token-level influences on model predictions. Figure 5.7 illustrates how the model distinguishes between AI-generated text (ChatGPT)

and human-authored content (Kantipur News).



(a) AI-generated sample (ChatGPT)



(b) Human-authored sample (Kantipur News)

Figure 5.7: SHAP token-level contributions. Red highlights indicate features contributing toward an *AI* classification, while Blue highlights indicate contributions toward a *Human* classification. Darker shades represent stronger contributions.

The visualization reveals distinct linguistic patterns used by the classifier:

- **AI Indicators (Red):** As seen in Figure 5.7a, AI-generated text sample(ChatGPT) shows strong red contributions
- **Human Indicators (Blue):** In contrast, the text sample taken from Kantipur News Figure 5.7b shows strong blue contributions.

5.5 Key Findings

- **Best Performing Model:** Ensemble methods achieved the highest performance, with Advanced Stacking (Gradient Boosting Tree) reaching the best accuracy of 0.9853, outperforming both traditional machine learning and transformer-based models.
- **Traditional ML Performance:** Among traditional machine learning models, SVM (Linear Kernel) performed the best, achieving an accuracy of 0.9616, making it a strong baseline model.

- **Transformer Model Insights:** Transformer-based models demonstrated strong performance, with XLM-Roberta (base) achieving the highest accuracy of 0.9728 under full fine-tuning at 512 tokens. Additionally, increasing token length consistently improved model performance across all transformer models, with average accuracy rising from 0.9253 at 128 tokens to 0.9706 at 512 tokens.
- **Ensemble Advantage:** Combining multiple models through ensemble techniques significantly improved performance beyond both traditional ML and transformer baselines, with all ensemble models exceeding 98% accuracy, demonstrating better generalization and robustness.
- **Model Explainability:** SHAP analysis revealed what part of the text affected the model predictions.

5.6 Limitations

While our results are promising, there are limitations to this study. One major limitation is the size and scope of the datasets used. Although Nepali news and wikipedia data provided a broad range of text samples, a larger and more diverse text samples from other sources could further validate the findings and enhance the generalizability of the models. Additionally, the study predominantly focused on ChatGPT and Gemini generated text, which limits the applicability of our conclusions to other LLMs. Paraphrasing presents another limitation of the proposed model. In practical settings, AI-generated text is often modified or paraphrased to appear more human-like. Such transformations can alter surface-level linguistic patterns that the model relies on, making detection more challenging.

5.7 DISCUSSION

The results indicate that traditional machine learning models, particularly SVC with a linear kernel (accuracy: 0.9616), perform strongly as baseline approaches. This suggests that the distinction between AI-generated and human-written Nepali text can be partially captured using linear decision boundaries and well-engineered features.

Transformer-based models further improved performance by effectively capturing con-

textual and semantic relationships within the text. Among them, XLM-Roberta (base) achieved the highest accuracy of 0.9728 under full fine-tuning at 512 tokens, followed closely by Multilingual MiniLM-L12-H384 (0.9715) and NepBERTa (0.9677). The consistent improvement in accuracy with increasing token length from an average of 0.9253 at 128 tokens to 0.9706 at 512 tokens demonstrates that longer contextual information plays a significant role in distinguishing writing styles.

The most notable improvement was achieved through ensemble learning techniques, with all ensemble models surpassing 98% accuracy. The Advanced Stacking (Gradient Boosting Tree) model achieved the best overall performance at 0.9853, followed by Bagging Decision Tree (0.9841) and Gradient Boosting Classifier (0.9833). By combining predictions from multiple models, ensemble methods effectively reduced individual model biases and improved generalization, highlighting the complementary strengths of different models in capturing various linguistic patterns.

The SHAP explainability analysis provided additional insight into the decision-making process of the classifier.

Ultimately, while these results are localized to our specific dataset, they provide a blueprint for low-resource language detection. The interplay between Deep Learning for nuance and Ensemble methods for stability, backed by XAI for validation, creates a framework that is both high-performing and, perhaps more importantly, intellectually defensible.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

This research presented a comprehensive approach to detect AI-generated Nepali text, addressing a critical and underexplored challenge in the context of low-resource languages. A complete pipeline was developed, including the creation of data sets, preprocessing, feature extraction, model training, evaluation, and explainable AI integration.

The experimental phase yielded several vital insights. Surprisingly, traditional machine learning wasn't entirely overshadowed; the SVM with a linear kernel established a formidable baseline with an accuracy of 0.9616. This suggests that even linear boundaries can distinguish the structure of machine-authored Nepali script. However, the true depth of Nepali syntax was best captured by Transformer-based architectures. Among these, XLM-Roberta (base) emerged as a standout, reaching 0.9728 accuracy when fully fine-tuned at a 512-token window.

The performance was further improved through ensemble strategies. By leveraging an Advanced Stacking (Gradient Boosting Tree) approach, we achieved a peak accuracy of 0.9853. This success proves that combination of different models, each compensating for the others' biases, is far more reliable than any single architecture.

Furthermore, the integration of SHAP provided the necessary "why" behind the "what."

Ultimately, this study offers a robust, interpretable framework that proves high-accuracy detection is possible even for linguistically complex, low-resource languages like Nepali.

6.2 Future Enhancements

Although the proposed system achieved strong performance, there are several areas for future improvement and extension.

One potential enhancement is the expansion of the dataset to include more diverse and balanced samples from different domains, writing styles, and AI models. Incorporating data from newer and more advanced generative models would improve the robustness

of the system.

Another direction is the exploration of more advanced transformer architectures and larger pre-trained language models. Models specifically trained for low-resource languages like Nepali could further enhance performance.

Further improvements in explainability can also be made by integrating more advanced interpretability techniques to better understand model decisions at both the global and local levels.

As AI-generated text becomes increasingly sophisticated and harder to distinguish, future systems should be evaluated and trained against adversarially crafted text content deliberately designed to evade detection. Building robustness against such cases is essential for long-term reliability of the detection system.

Finally, future work can explore multilingual and cross-lingual detection, allowing the system to generalize beyond Nepali and detect AI-generated content in multiple languages.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[2] R. M. Schmidt, "Recurrent neural networks (rnns): A gentle introduction and overview," 2019. [Online]. Available: https://arxiv.org/abs/1912.05911

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018. [Online]. Available: https://www.mikecaptain.com/resources/pdf/GPT-1.pdf

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[6] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[7] S. Khanuja, D. Bansal, S. Mehtani, S. Khosla, A. Dey, B. Gopalan, D. K. Margam, P. Aggarwal, R. T. Nagipogu, S. Dave, S. Gupta, S. C. B. Gali, V. Subramanian, and P. Talukdar, "Muril: Multilingual representations for indian languages," 2021.

[8] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023. [Online]. Available: https://arxiv.org/abs/2302.13971

[9] S. Timilsina, M. Gautam, and B. Bhattarai, "NepBERTa: Nepali language model trained in a large corpus," in *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 2:*

Short Papers), Y. He, H. Ji, S. Li, Y. Liu, and C.-H. Chang, Eds. Online only: Association for Computational Linguistics, Nov. 2022, pp. 273–284. [Online]. Available: https://aclanthology.org/2022.aacl-short.34/

[10] S. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," 2017. [Online]. Available: https://arxiv.org/abs/1705.07874

[11] B. Li, Z. Zhao, T. Liu, P. Wang, and X. Du, "Weighted neural bag-of-n-grams model: New baselines for text classification," in *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, 2016, pp. 1591–1600.

[12] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in *International conference on machine learning*. PMLR, 2023, pp. 17 061–17 084.

[13] P. He, X. Liu, J. Gao, and W. Chen, "Deberta: Decoding-enhanced bert with disentangled attention," 2021. [Online]. Available: https://arxiv.org/abs/2006.03654

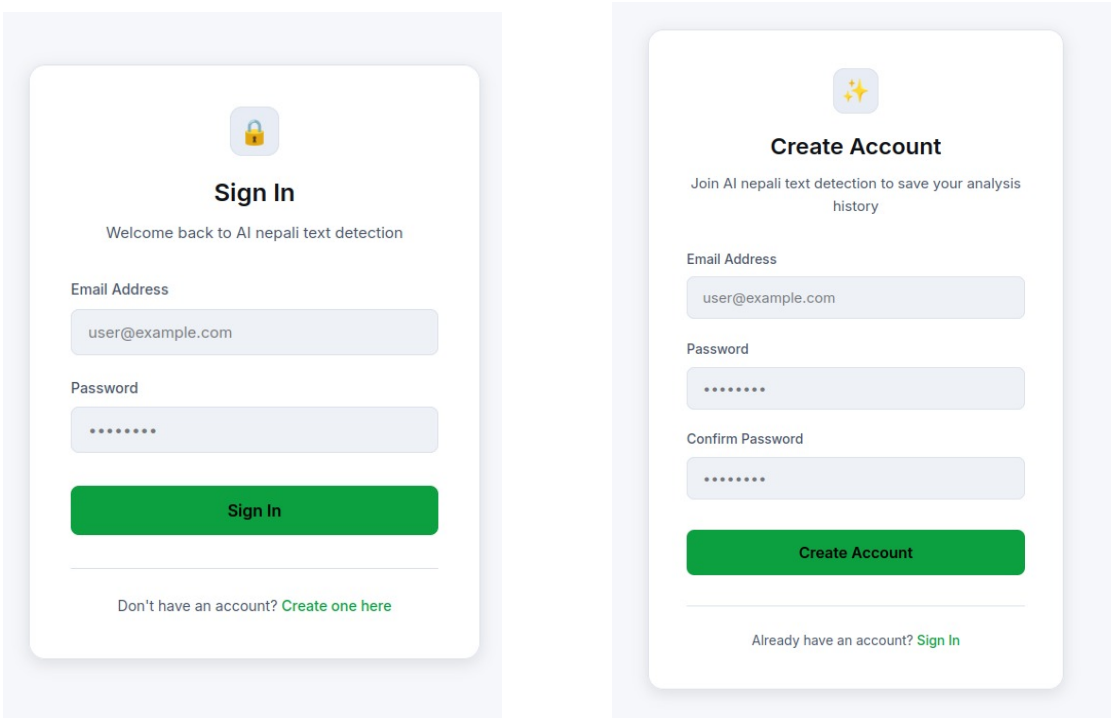
[14] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, 2020.

[15] C. Chen and K. Shu, "Can llm-generated misinformation be detected?" *arXiv preprint arXiv:2309.13788*, 2023.

[16] K. Yalcin, I. Cicekli, and G. Ercan, "An external plagiarism detection system based on part-of-speech (pos) tag n-grams and word embedding," *Expert Systems with Applications*, vol. 197, p. 116677, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417422001610

[17] M. T. Ribeiro, S. Singh, and C. Guestrin, "" why should i trust you?" explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.

Appendix A



(a) Sign In Page (b) Sign Up Page

Figure 6.1: Authentication screens of the application.

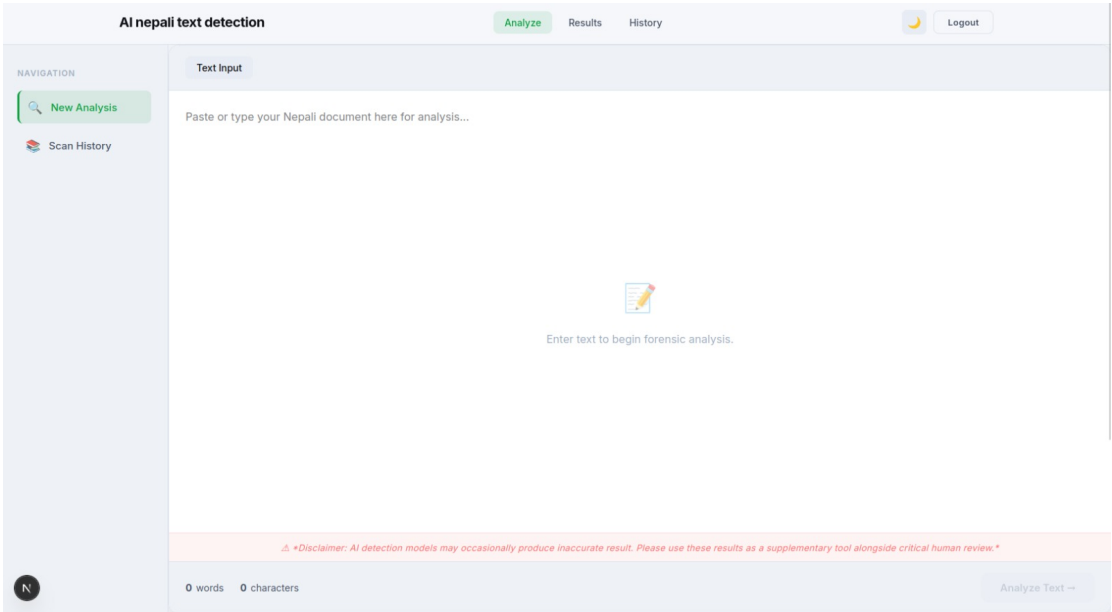


Figure 6.2: Input Page

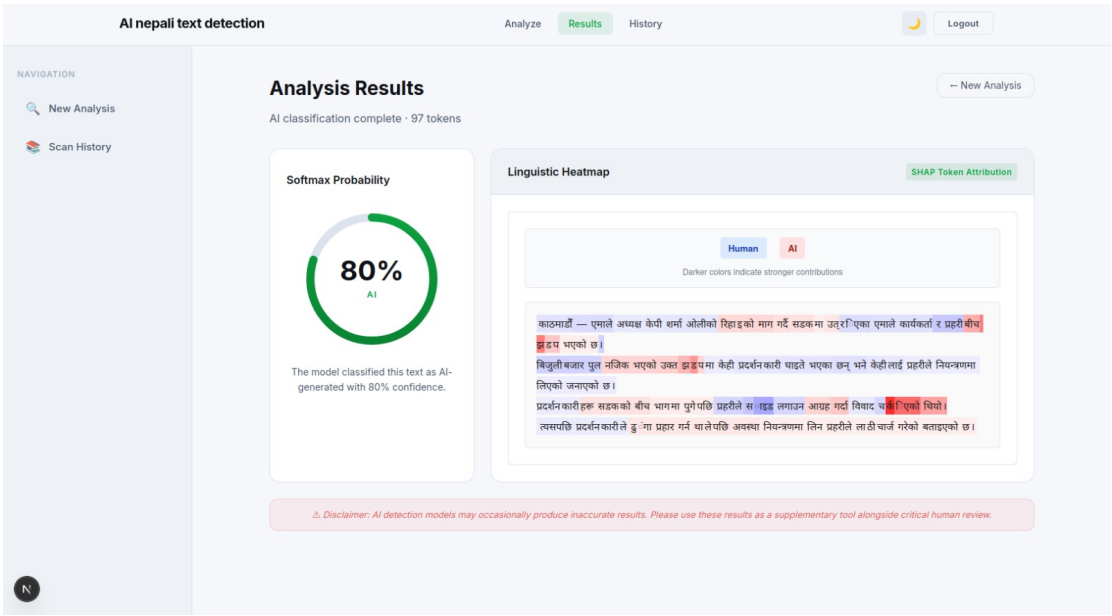


Figure 6.3: Results Page

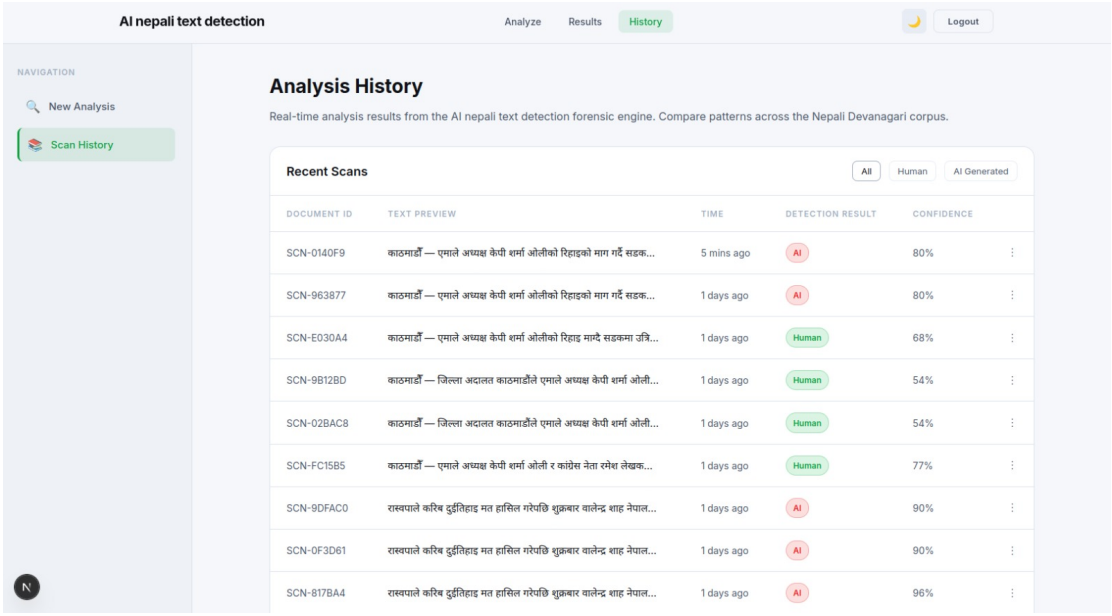


Figure 6.4: History Page

Appendix B

	prompt_type	text	category	label	ai_model
0	generate	नेपाल जुन कुनै समय दुर्गम क्षेत्रका रूपमा परिच...	Blog	ai	gemini-2.5
1	None	मीनराज भण्डारीकाठमाडौँ असोज २४ गते ।दसैँ नजिकि...	News	human	None
2	None	काठमाडौँ असार १३ गते ।माध्यमिक शिक्षा परीक्षा ...	News	human	None
3	None	काठमाडौँ — सन् १९७३ को ओस्कर अवार्डमा ससिन लिट...	Entertainment	human	None
4	write_nepali	जलवायु परिवर्तन वर्तमान युगको सबैभन्दा ठूलो र ...	World	ai	gemini-2.5
5	complete	बुवा दिवस पितालाई सम्मान दिने हिन्दू परम्परागत...	Wikipedia	ai	gpt-5-mini
6	polish	काठमाडौँ — एसीसी यू–एसिया कपका लागि देव खनालको...	Sports	ai	gpt-5-mini
7	formal	नेपाली क्रिकेट टोलीले हालै सम्पन्न विश्वकप छनो...	Sports	ai	gpt-5-mini
8	expand	राज्यशक्तिको प्रयोग बहु तहका सरकारहरूबाट गर्ने...	News	ai	gemini-3.1-flash
9	write_nepali	नेपाली चलचित्र जगतले पछिल्लो समयमा ठूलो फड्को ...	Entertainment	ai	gemini-3.1-flash

Figure 6.5: Sample Dataset Snippet



Figure 6.6: Human Data Wordcloud



Figure 6.7: AI Data Wordcloud